DAY 6 TASKS — USER AUTHENTICATION SYSTEM

D. Vishnu Vardhan Reddy

donapativishnu21@gmail.com

Introduction

The objective of this project is to develop a secure backend authentication system using Node.js, Express.js, MongoDB, bcrypt, and JSON Web Token (JWT). The system allows users to register, log in securely, access protected routes, and perform CRUD operations with proper validations. API testing is performed using Postman to verify the functionality of all backend APIs.

Task 1 — User Authentication System

Objective

The purpose of this task is to create a complete user authentication module. The authentication system ensures that user credentials are stored securely, and only authenticated users can access protected resources.

Technologies Used

The following technologies are used in this project:

- Node.js
- Express.js
- MongoDB
- Mongoose

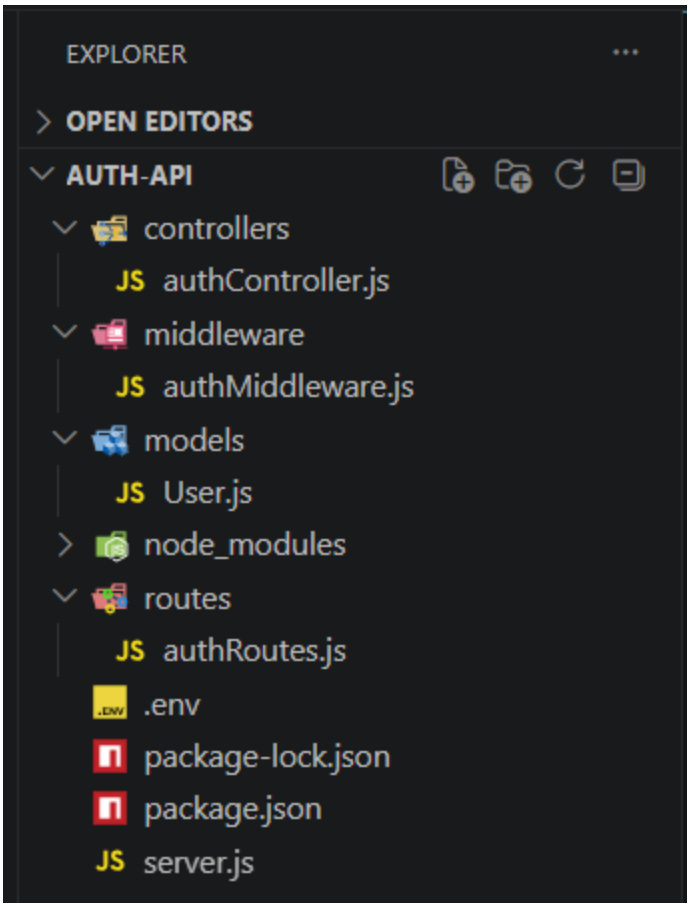
- bcrypt
- JWT (JSON Web Token)
- Postman

Step 1 — Create Backend Project

First, create a new backend project folder and initialize a Node.js application using npm. Install the required packages such as Express.js, Mongoose, bcrypt, jsonwebtoken, dotenv, and nodemon.

These packages help in:

- creating the server,
- connecting MongoDB,
- hashing passwords,
- generating authentication tokens,
- and managing environment variables.



Step 2 — Setup Express Server

An Express server is created in the `server.js` file. Middleware such as `express.json()` is used to accept JSON data from API requests. MongoDB is connected using Mongoose.

The server runs on port 5000 and listens for incoming client requests.

JS server.js > ...

```
const express = require("express");
const mongoose = require("mongoose");
require("dotenv").config();

const authRoutes = require("./routes/authRoutes");
const authMiddleware = require("./middleware/authMiddleware");

const app = express();

app.use(express.json());

mongoose.connect(process.env.MONGO_URI)
.then(() => console.log("MongoDB Connected"))
.catch((err) => console.log(err));

app.get("/", (req, res) => {
    res.send("Server Running");
});

app.use("/auth", authRoutes);


// PROTECTED ROUTE
app.get("/profile", authMiddleware, (req, res) => {

    res.status(200).json({
        message: "Welcome to Profile",
        user: req.user
    });

});

```

Step 3 — Create User Model

A User model is created using Mongoose Schema. The schema contains the following fields:

- name
- email
- password

The email field is unique to prevent duplicate user registrations. The password field stores encrypted passwords instead of plain text passwords.

models > JS User.js > ...

```
const mongoose = require("mongoose");

const userSchema = new mongoose.Schema({

    name: {
        type: String,
        required: true
    },

    email: {
        type: String,
        required: true,
        unique: true
    },

    password: {
        type: String,
        required: true
    }

});

module.exports = mongoose.model("User", userSchema);
```

Step 4 — Create Register API

A Register API is created using the POST method.

API Endpoint

`/auth/register`

The API accepts user details such as:

- name
- email
- password

Before storing the user data in MongoDB:

- the system checks whether the email already exists,
- the password is encrypted using bcrypt hashing.

After successful registration, a success message is returned to the user.

Step 5 — Password Hashing using bcrypt

bcrypt is used to encrypt passwords before storing them in the database.

Password hashing improves security because:

- original passwords are hidden,
- stolen database data cannot reveal actual passwords,
- user credentials remain protected.

The hashed password is stored in MongoDB instead of the original password.

Step 6 — Create Login API

A Login API is created using the POST method.

API Endpoint

`/auth/login`

The Login API accepts:

- email
- password

The system performs the following operations:

1. Checks whether the user exists.
2. Compares the entered password with the hashed password using bcrypt.
3. Generates a JWT token after successful login.

If login is successful, the server returns:

- success message

- JWT token

If login fails, proper error messages are returned.

Step 7 — JWT Token Generation

JWT (JSON Web Token) is generated after successful login.

The token contains encoded user information and is used for authentication in protected routes.

JWT improves backend security because:

- users are verified before accessing protected resources,
- unauthorized users are blocked,
- sessions can be managed securely.

The generated token is sent to the client and stored for future authenticated requests.

Task 2 — Protected Route Implementation

Objective

The purpose of this task is to create protected routes using JWT middleware.

Protected routes can only be accessed by authenticated users with valid JWT tokens.

Step 1 — Create Authentication Middleware

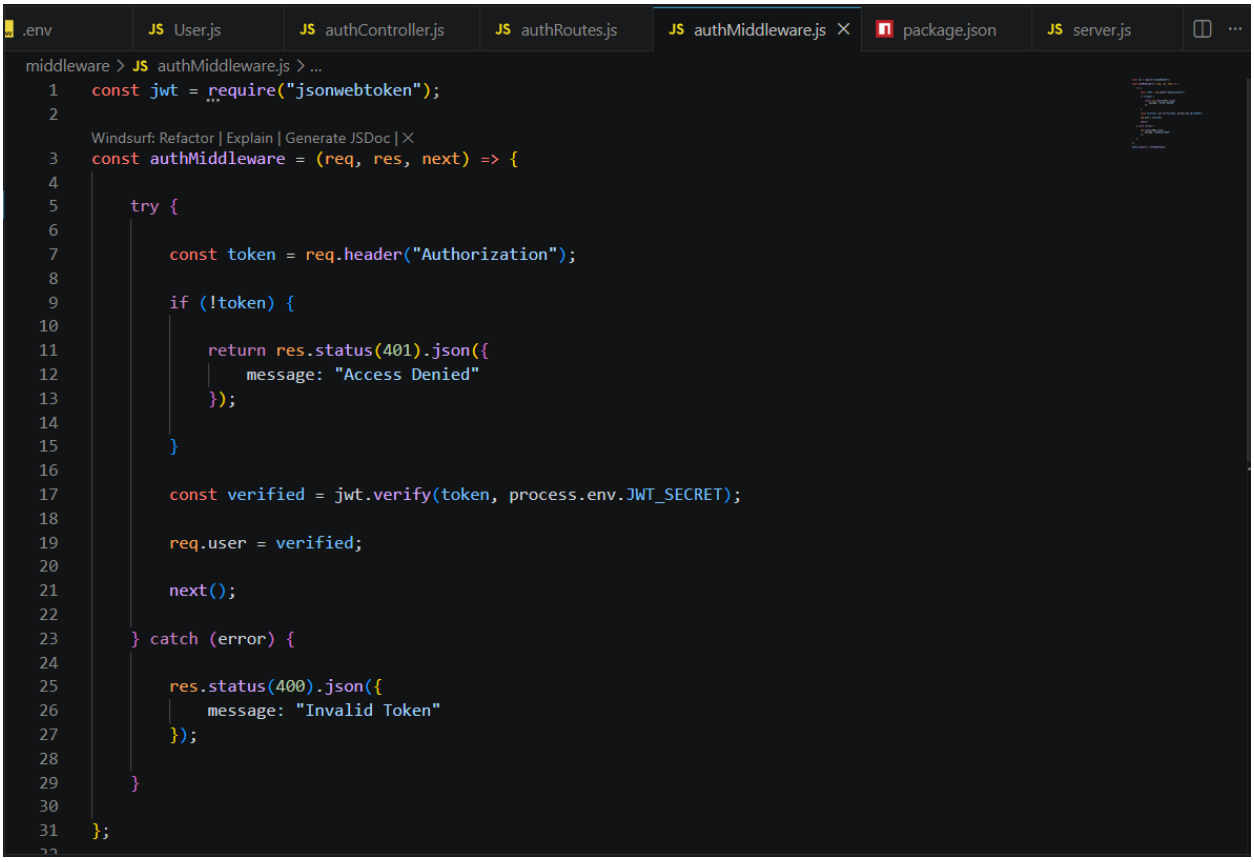
Authentication middleware is created to verify JWT tokens.

The middleware:

- extracts the token from request headers,
- verifies the token using JWT secret key,
- allows access only if the token is valid.

If the token is missing or invalid:

- access is denied,
- an error message is returned.



Step 2 — Create Protected API

A protected API is created using the GET method.

API Endpoint

`/profile`

Only authorized users with valid JWT tokens can access this route.

Unauthorized users receive:

- access denied message,
- invalid token error.

Task 3 — CRUD API Enhancement

Objective

The purpose of this task is to improve CRUD APIs with validations and proper response handling.

Step 1 — Add Field Validations

The APIs validate required fields before storing data in MongoDB.

The validation checks:

- empty fields,
- missing values,
- invalid submissions.

If validation fails, appropriate error messages are returned.

Step 2 — Handle Database Errors

Database operations are wrapped inside try-catch blocks to handle runtime errors properly.

The server returns:

- proper status codes,
- structured JSON responses,
- meaningful error messages.

This improves API stability and debugging.

Step 3 — Structured JSON Responses

All APIs return properly formatted JSON responses.

Example:

- success messages,
- error messages,
- returned data objects.

Structured responses improve frontend integration and API readability.

Task 4 — API Testing using Postman

Objective

The purpose of this task is to test all backend APIs using Postman.

APIs Tested

The following APIs are tested:

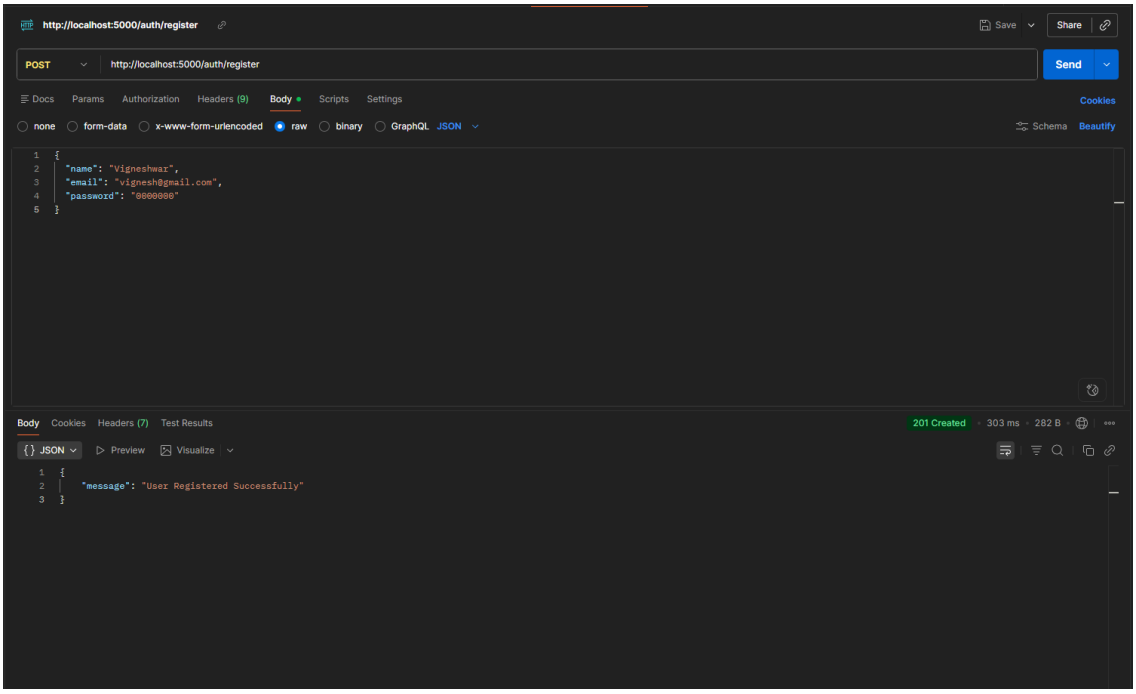
- Register API
- Login API
- Protected Route API
- CRUD APIs

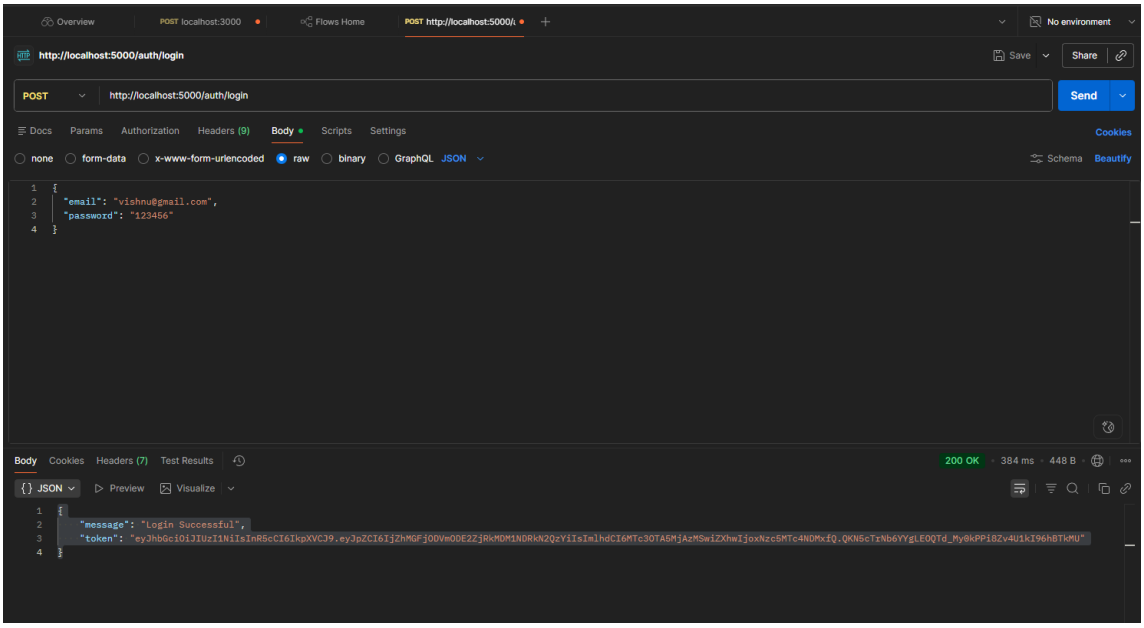
Postman Testing Process

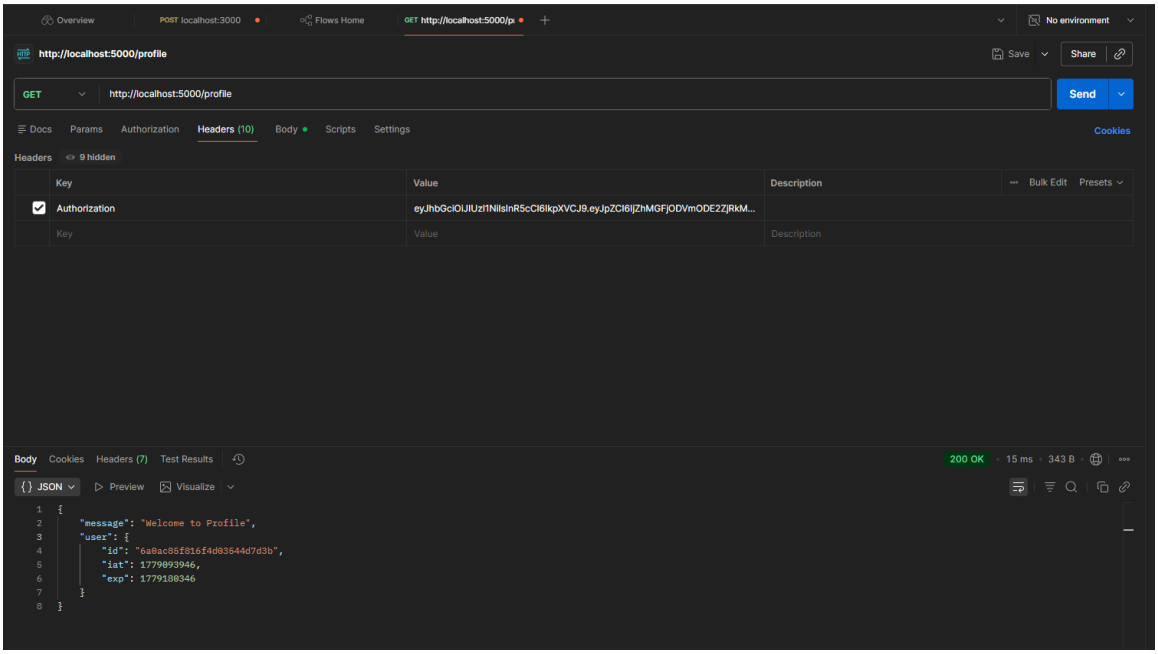
Each API request is tested by:

- sending request data,
- verifying server responses,
- checking status codes,
- validating JWT tokens.

Screenshots of successful API responses are captured as deliverables.


http://localhost:5000/auth/register
Save
Share
POST
http://localhost:5000/auth/register
Send
Docs Params Authorization Headers (9) Body Scripts Settings
Cookies
none form-data x-www-form-urlencoded raw binary GraphQL JSON
Schema Beautify
{
"name": "Vigneshwar",
"email": "vignesh@gmail.com",
"password": "0000000"
}
Body Cookies Headers (7) Test Results
201 Created 303 ms 282 B
{ } JSON Preview Visualize
{
"message": "User Registered Successfully"
}


Overview
POST localhost:3000
Flows Home
POST http://localhost:5000/
No environment
http://localhost:5000/auth/login
Save
Share
POST
http://localhost:5000/auth/login
Send
Docs Params Authorization Headers (9) Body Scripts Settings
Cookies
none form-data x-www-form-urlencoded raw binary GraphQL JSON
Schema Beautify
{
"email": "vishnu@gmail.com",
"password": "123456"
}
Body Cookies Headers (7) Test Results
200 OK 384 ms 448 B
{ } JSON Preview Visualize
{
"message": "Login Successful",
"token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjZhMGFjODVmODE2ZjRkMDM1NDRkN2QzYiIsImlhdCI6MTc3OTA5MjAzMSwiZXhwIjoxNzc5MTc4NDMxfQ.QKN5cTrNb6YYgLEOQTd_My0kPPi8Zv4U1kI96hBTkMU"
}



Expected Output

After successful implementation:

- User data is stored securely in MongoDB.
- Passwords are encrypted using bcrypt.
- JWT tokens are generated successfully.
- Protected routes allow only authorized users.
- CRUD APIs work with validations.
- Proper error messages are returned.
- All APIs are tested successfully using Postman.

Conclusion

In this project, a complete authentication and authorization system was developed using Node.js, Express.js, MongoDB, bcrypt, and JWT. Secure user registration and login functionalities were implemented with encrypted password storage and token-based authentication. Protected routes were created to restrict unauthorized access. CRUD APIs were enhanced with validations and structured responses. Finally, all backend APIs were tested successfully using Postman.

GitHub link

https://github.com/vishnuvardhan15/auth-api.git